

Azure Data Engineering & Architecture Handbook

One Enterprise Project from Platform Foundations to Data, Applications, AI and Operations

Target outcome: Be able to take an enterprise requirement, choose the right Azure components, explain how identity/network/compute/data/application layers connect, integrate Databricks and Snowflake, and defend the design using clear Principal/Architect language.

This handbook assumes strong software/data engineering experience and familiarity with Databricks. Azure-specific concepts are explained simply first, then connected to enterprise architecture decisions. Databricks comparisons are used as shortcuts where useful, but they are not treated as exact equivalents.

Contents

- 1. The Azure Mental Model - Subscription to Running Workload
- 2. The Anchor Project - Customer 360 & AI Engagement Platform
- 3. Identity and Access - Entra ID, Service Principals, Managed Identity, RBAC
- 4. Networking - VNet, Subnet, DNS, Private Endpoint, Front Door, App Gateway, Load Balancer
- 5. Compute - Functions, App Service, Container Apps, AKS, VMs and Serverless
- 6. Storage and Databases - ADLS, OneLake, Azure SQL, PostgreSQL, Cosmos DB, Cache
- 7. Messaging and Events - Service Bus, Event Grid, Event Hubs
- 8. Azure Data Factory - Deep Enterprise Patterns
- 9. Microsoft Fabric - Data Factory, OneLake, Lakehouse, Warehouse, Spark, Real-Time
- 10. Synapse - Dedicated SQL, Serverless SQL, Spark and Existing Enterprise Estates
- 11. Databricks and Snowflake Integration in Azure
- 12. Power BI and the Serving Layer
- 13. Azure Machine Learning - Compute, Environments, Pipelines and Endpoints
- 14. Azure OpenAI, AI Search, SharePoint RAG and Agentic Patterns
- 15. Control-M and Cross-Platform Orchestration
- 16. Security, Secrets, Private Connectivity and Enterprise Guardrails
- 17. CI/CD, IaC, Monitoring, Reliability, DR and FinOps
- 18. End-to-End Project Build
- 19. Component Selection and Trade-off Playbook
- 20. Architect Communication and Interview Answers

1. The Azure Mental Model - Subscription to Running Workload

Azure becomes much easier when you stop memorizing service names and instead ask six questions: **Where does it live? What runs it? Who is it? What can it access? How does it connect? How is it operated?**

Tenant	->	Subscription	->	Resource Group	->	Resource	->	Identity	->	Network
--------	----	--------------	----	----------------	----	----------	----	----------	----	---------

1.1 Tenant, subscription, resource group and resource

Microsoft Entra tenant is the identity boundary containing users, groups, applications and service identities. A **subscription** is a billing/governance boundary under which Azure resources are created. A **resource group** is a lifecycle/management container for related resources. A **resource** is the actual service instance: storage account, Function App, Key Vault, VNet, database and so on.

How an architect says it: “I separate identity, billing/governance and application lifecycle boundaries. The Entra tenant answers who; subscriptions help isolate ownership, policy and cost; resource groups group resources that are operated together.”

1.2 Resource provider, region and availability zone

Azure services are supplied through resource providers. Region selection affects latency, data residency, service availability and disaster recovery. Availability Zones are physically separated locations within a region for supported services. Not every service or SKU has identical zone/region behavior.

1.3 The platform-access chain

Code	->	Compute	->	Identity	->	RBAC/data permission	->	Network path	->	Target service
------	----	---------	----	----------	----	----------------------	----	--------------	----	----------------

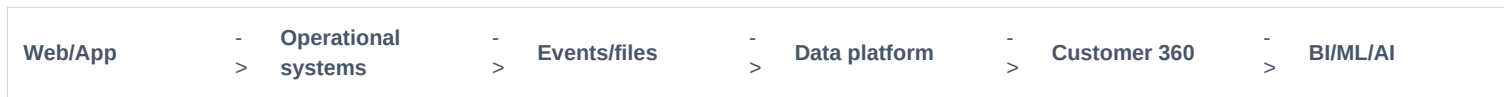
Remember this chain: My code runs somewhere. That runtime has an identity. The identity receives permissions. The runtime still needs a network path to the target. Authentication and networking solve different problems.

How an architect says it: “Before I debug the Python, I verify runtime, identity, authorization and connectivity. A correct token does not help if private DNS/networking is wrong, and a reachable endpoint does not help if the workload identity lacks permission.”

2. Anchor Project - Enterprise Customer 360 & AI Engagement Platform

2.1 What Customer 360 means

Customer 360 is not an Azure product. It is the enterprise capability to resolve customer identity and combine trusted information - profile, loyalty, purchases, interactions, digital behavior, preferences and service history - into a governed view that can support analytics, segmentation, ML, personalization and AI.



2.2 Project requirements

- Web/mobile customers place orders and update profiles.
- Operational order data is relational; profile/preferences can be document-oriented where justified.
- Store and digital events arrive continuously; vendor files arrive daily; some legacy sources remain on-premises.
- BI requires governed customer, sales and loyalty analytics.
- ML requires churn/next-best-action features.
- A chatbot answers policy/account questions using SharePoint and governed customer data.
- Critical customer-facing APIs require high availability; analytics has different RPO/RTO.
- PII must be privately accessed, audited and least-privilege.
- Enterprise scheduling already uses Control-M.
- Databricks is strong in engineering; some domains may use Snowflake; Power BI is a major consumer.

2.3 Target architecture at a glance



Around that flow sit Entra ID, Managed Identity/service principals, Key Vault, VNets/private endpoints/DNS, Azure Monitor/Application Insights, CI/CD and infrastructure-as-code. The book repeatedly returns to this project so each service has a reason to exist.

3. Identity and Access

3.1 Authentication vs authorization

Authentication proves who/what you are. **Authorization** decides what that identity may do. Entra ID provides identity/token capabilities; Azure RBAC and service-specific permissions govern actions/data.

3.2 Service principal

A service principal is an application identity in Entra ID. It is useful for non-human workloads, automation and cross-service authentication. Credentials may be certificate/secret/federated identity depending on pattern. Long-lived client secrets create rotation and leakage risk.

3.3 Managed Identity

Managed Identity gives an Azure-hosted workload an Entra identity whose credential lifecycle is managed by Azure. The workload obtains tokens without storing a client secret in code. System-assigned identity is tied to one resource lifecycle; user-assigned identity is a reusable identity resource.

How an architect says it: “If the workload runs on Azure and the target supports Entra authentication, Managed Identity is my default because it removes application-managed credentials. I still grant only the minimum data-plane permissions required.”

3.4 RBAC and data-plane permissions

Azure RBAC controls actions on Azure resources, but many services also have data-plane permissions. For example, being able to view a storage account resource is not the same as being allowed to read blobs. Always distinguish management-plane and data-plane access.

Architecture decision

Requirement: ADF/Function needs to read a private ADLS container.

Recommendation: Use its Managed Identity and grant the minimum storage data role at the narrowest practical scope.

Why: No embedded storage key; auditable identity-based access.

Trade-off: Role design and private networking still need setup.

Alternative: Service principal where workload/tenant constraints require it.

4. Networking - The Part That Makes Code 'Mysteriously' Fail

4.1 VNet and subnet

A Virtual Network is a private IP network boundary in Azure. Subnets divide it into smaller address ranges for workload separation and routing/security. Services may be deployed into a VNet, integrated with one, or accessed privately through endpoints depending on the service.

4.2 Private Endpoint

A Private Endpoint gives a supported Azure service a private IP in your VNet. Clients use that private path instead of reaching the public service endpoint. Private DNS is commonly required so the normal service hostname resolves to the private IP from the correct network.

How an architect says it: "Private Endpoint is about private network reachability. It does not replace identity. I need both the private route/DNS and authorization to the target service."

4.3 NSG, firewall and routing

Network Security Groups filter traffic at subnet/network-interface boundaries. Service firewalls restrict allowed network sources. Route tables influence where traffic flows. Enterprise designs often centralize ingress/egress and DNS in hub-spoke networks.

4.4 Front Door vs Application Gateway vs Load Balancer

Component	Layer / scope	Use it when
Azure Front Door	global HTTP(S) edge	global web/API entry, edge routing, acceleration, WAF
Application Gateway	regional Layer 7 HTTP(S)	regional web routing, TLS/WAF, path/host rules into VNet workloads
Azure Load Balancer	Layer 4 TCP/UDP	network-level load balancing without HTTP-aware routing
API Management	API gateway, not generic load balancer	API policies, auth, quotas, versioning, developer/API governance

Architecture decision

Requirement: Global customer API with WAF and governed APIs.

Recommendation: Front Door at global edge, API Management for API governance, then application compute.

Why: They solve different layers: edge traffic/security versus API lifecycle/policies.

Trade-off: More hops, cost and operational configuration.

Alternative: For a small regional internal API, simpler ingress may be enough.

5. Compute - Where Does My Code Actually Run?

5.1 Compute selection

Service	Mental model	Strong fit	Trade-off
Azure Functions	event/serverless function	short event/API automation, bursty tasks	execution/runtime limits and cold/start behavior; avoid heavy long jobs
App Service	managed web/app hosting	web apps and APIs with minimal platform ops	less container/orchestrator control than AKS
Container Apps	managed containers + scaling	containerized APIs/workers/events without full Kubernetes ownership	less low-level control than AKS
AKS	managed Kubernetes control plane	complex container platforms, custom networking/scheduling, many services	highest platform/operational complexity
VM	server you manage	legacy/special software or OS-level control	patching, scaling, availability are your responsibility

5.2 Databricks analogy

Databricks abstracts much of Spark compute selection for data workloads. Azure application compute is broader: you choose the runtime based on request/event model, packaging, scaling, networking and operational ownership. AKS is not 'better compute'; it is more control with more platform responsibility.

5.3 Functions in data engineering

Functions are useful for lightweight custom logic around a pipeline: webhook receiver, event handler, token/API helper, validation, metadata operation or small transformation. They should not become a dumping ground for large ETL that belongs in a data engine.

Architecture decision

Requirement: API requires custom signing and pagination logic before landing data.

Recommendation: Use ADF for orchestration/copy where possible and a Function for the small custom API behavior.

Why: Keeps orchestration declarative while isolating code that ADF is poor at expressing.

Trade-off: Adds another deployable component.

Alternative: A containerized extractor if logic is long-running/heavy.

6. Storage and Operational Databases

6.1 ADLS Gen2

ADLS Gen2 is Azure object storage optimized for hierarchical data-lake workloads. It commonly holds raw/curated files and open table data used by Databricks, Synapse and other engines. Storage account, container/filesystem, path, identity and network access are separate design concerns.

6.2 OneLake

OneLake is the tenant-wide logical storage foundation of Microsoft Fabric. Fabric items such as Lakehouses and Warehouses use OneLake-backed storage abstractions. Shortcuts can reference data without traditional copy patterns in supported scenarios.

6.3 Azure SQL vs PostgreSQL vs Cosmos DB

Need	Azure SQL	Azure Database for PostgreSQL	Cosmos DB
Model	relational/T-SQL	relational/PostgreSQL ecosystem	distributed NoSQL/document/key-value APIs depending on model
Transactions/joins	strong relational fit	strong relational fit	design around partition/key access; cross-partition patterns need care
Choose when	SQL Server compatibility/Microsoft relational stack	Postgres/open ecosystem/extensions/app preference	massive scale, low-latency distributed access, flexible document model
Architect watch	service tier, HA, connection scaling	HA, extensions, connection pooling	partition key, RU/throughput model, consistency, hot partitions

6.4 Cosmos partition key

Cosmos DB scales data and request processing across logical/physical partitions. The partition key therefore becomes a fundamental architecture decision. A poor key can create hot partitions, cross-partition queries and uneven cost. Choose a key that distributes load while supporting dominant access patterns.

Architecture decision

Requirement: Order service needs relational transactions across order header/lines and reporting joins.

Recommendation: Use PostgreSQL or Azure SQL as the operational source, based on ecosystem/compatibility.

Why: Relational constraints and transaction semantics fit naturally.

Trade-off: Scale model differs from globally distributed document store.

Alternative: Cosmos DB if access is naturally partitioned documents and global low-latency scale dominates.

6.5 Cache

A distributed cache such as Azure Cache for Redis can reduce repeated database/API reads for hot data and sessions. Cache introduces invalidation, consistency and failure-mode questions. It is a performance component, not the source of truth.

7. Messaging and Events

Service	Think of it as	Use when
Service Bus	enterprise queue/topic messaging	commands/work queues, durable delivery, sessions/order, dead-letter, pub-sub
Event Grid	event notification/routing fabric	something happened; route events to handlers
Event Hubs	high-throughput event stream/log	telemetry/clickstream/large continuous event ingestion
Storage Queue	simple queue	basic asynchronous decoupling without richer broker features

7.1 Service Bus

Queues decouple producer and consumer. Topics/subscriptions support pub-sub. Production patterns include lock/settlement behavior, retries, dead-letter queues, duplicate detection where appropriate, sessions/order requirements and idempotent consumers.

7.2 Event Grid

Event Grid is suited to reactive notifications: blob created, resource event, application event. It routes events to subscribers; it is not the same as a high-volume replayable analytics stream.

7.3 Event Hubs

Event Hubs is designed for high-throughput event ingestion. Events are partitioned; consumers track offsets/checkpoints and can use consumer groups. Partition count/key affects parallelism and ordering. Retention enables replay within the retained window.

How an architect says it: “I use Service Bus when the message represents work that must be processed reliably, Event Grid when I need event notification/routing, and Event Hubs when I need a high-throughput ordered-partition event stream for analytics or telemetry.”

8. Azure Data Factory - Deep Enterprise Patterns

8.1 Core objects

ADF concept	Simple meaning	Databricks comparison
Pipeline	workflow/orchestration container	Lakeflow Job workflow conceptually
Activity	one step	task
Linked Service	connection/configuration to a service	connection/credential concept, not a table
Dataset	named representation of data used by activities	input/output dataset reference
Integration Runtime	compute/connectivity used for movement/activity execution	no exact equivalent; think execution + network bridge
Trigger	starts a pipeline	schedule/event trigger
Parameter	runtime input	job/task parameter

8.2 Integration Runtime

Azure Integration Runtime provides managed cloud connectivity/compute for supported data movement and activity dispatch. **Self-hosted Integration Runtime** runs on infrastructure you control and is commonly used to reach private/on-premises sources. Azure-SSIS IR exists for lifting/running SSIS packages where that migration strategy is required.

For self-hosted IR, architecture includes host placement, outbound connectivity, source reachability, HA nodes, throughput, patching, monitoring and credential handling. The IR is not merely an ADF checkbox; it is part of the network path.

How an architect says it: “The source is on-premises, so the first constraint is connectivity. I would place a highly available self-hosted IR where it can reach the source and Azure control/data endpoints, validate throughput, then land into ADLS before downstream processing.”

8.3 Copy Activity

Copy Activity moves data between supported sources/sinks. Design concerns include source query/predicate, parallel copies, partitioned reads, staging, file format, compression, sink write behavior, fault tolerance, retries and throughput. For large database extraction, avoid an unbounded `SELECT *` if a partitioned/incremental extraction is possible.

8.4 Metadata-driven ingestion

A metadata-driven framework stores source table, extraction mode, watermark column, sink path/table, load options and validation rules in configuration. A controller pipeline reads configuration, iterates entities, executes reusable child pipelines and records audit status. This reduces duplicated pipelines but must not hide all logic in unreadable expressions.

```
Config      -> Lookup      -> ForEach      -> Copy/Invoke      -> Validate      -> Audit
```

8.5 Incremental watermark pattern

Store the last successfully committed source watermark. At run start, capture a stable upper bound. Extract rows in the interval, land/process them, validate and only then advance the stored watermark. Consider overlap + deduplication for timestamps, and do not advance state if downstream commit fails.

8.6 REST/API ingestion

ADF can call REST endpoints and copy paginated responses when the API pattern is supported. Enterprise concerns are authentication, token lifetime, pagination, rate limits, retries/backoff, cursor/watermark semantics, schema drift, raw-response retention and API contract changes. Use Functions/Container Apps when custom control flow becomes more natural in code.

8.7 ADF to Databricks, Snowflake and Synapse

ADF can orchestrate and move data around multiple platforms. A common enterprise pattern is to use ADF for source connectivity/landing and invoke Databricks for Spark/Delta transformations. Snowflake can be a source/sink and transformation platform. Synapse SQL can be loaded/queried through its connectors. The key architecture question is ownership: avoid duplicating orchestration logic across ADF and the target platform.

Architecture decision

Requirement: Nightly Oracle extract, transformation in Databricks, enterprise Control-M above both.

Recommendation: Control-M triggers ADF; ADF handles Oracle connectivity/landing; ADF invokes a Databricks job or signals the platform-local workflow.

Why: Clear responsibility: enterprise schedule -> source ingestion -> data-engine transformation.

Trade-off: Cross-platform observability and failure propagation must be designed.

Alternative: Move orchestration fully into one platform only if enterprise dependency requirements allow it.

9. Microsoft Fabric - Modern Data Platform Mental Model

9.1 Fabric in one sentence

Microsoft Fabric is an integrated SaaS analytics platform that brings ingestion, engineering, warehouse, real-time analytics, data science and Power BI experiences around OneLake and shared governance/administration.

9.2 Capacity and compute

Fabric is capacity-oriented: workloads consume capacity resources rather than every experience being managed like a separate Databricks cluster. Spark also has pool/session concepts for engineering workloads. The architect must understand capacity sizing, workload contention, concurrency and cost governance.

9.3 Lakehouse vs Warehouse

	Fabric Lakehouse	Fabric Warehouse
Primary style	Spark/lakehouse + SQL access over Delta-oriented data	SQL-first relational warehouse experience
Developer	data engineers/data scientists + SQL users	SQL/BI teams
Model	files/tables, medallion, engineering	dimensional/relational analytical serving
Choose when	Spark/open lakehouse processing is central	governed SQL warehouse semantics/BI are central

9.4 Data Factory in Fabric

Fabric Data Factory provides pipeline/data movement and Dataflow capabilities within Fabric. If you know ADF, the orchestration mental model is familiar, but evaluate current connector, gateway, deployment, monitoring and Fabric integration capabilities rather than assuming feature-for-feature identity.

9.5 Spark pools and sessions

Fabric Spark provides managed Spark execution. Starter/default pool capabilities reduce setup time; custom pools let teams tune node family/size, autoscale and other settings where supported. Compared with Databricks, focus on the same workload questions - data volume, shuffle, concurrency, startup, libraries and cost - then learn the Fabric-specific pool/capacity controls.

9.6 Real-Time Intelligence

Real-Time Intelligence addresses event ingestion, processing, exploration and operational analytics. Eventstream-style capabilities can ingest/route events; KQL-based stores/queries are useful for high-volume time/event data. This is different from simply running a five-minute batch pipeline.

9.7 When Fabric vs Databricks

How an architect says it: "I would favor Fabric when tight Power BI integration, SaaS simplicity, OneLake and a unified Microsoft analytics experience are major drivers. I would favor Databricks when Spark engineering depth, large-scale complex processing, lakehouse/ML engineering and its platform capabilities are central. In a real enterprise I also consider existing skills, governance and platform consolidation cost."

10. Azure Synapse - Understand the Existing Estate

10.1 Synapse components

Synapse brought together SQL analytics, Spark, pipelines/integration and workspace experiences. Many enterprises still operate Synapse assets even as Fabric changes the strategic Microsoft analytics conversation. An architect must understand what exists before recommending migration.

10.2 Dedicated SQL pool

Dedicated SQL pool is an MPP analytical warehouse. Compute nodes process distributed table data in parallel. Physical distribution is critical because joins/aggregations can require data movement between distributions.

Distribution	Use	Risk
Hash	large fact tables joined/grouped on stable key	skew if key distribution is poor
Round-robin	fast/simple landing when no good distribution key	later joins often require data movement
Replicated	small dimensions copied to compute nodes	not suitable for large frequently changing tables

Performance topics include distribution skew, data movement, columnstore behavior, statistics, partitioning, workload management/resource classes, concurrency, loading patterns and scaling/pause-resume.

How an architect says it: “In a dedicated pool I check distribution before I blame SQL syntax. A large fact-to-fact join on incompatible distribution keys can move enormous data across nodes.”

10.3 Serverless SQL

Serverless SQL queries data in the lake without provisioning a dedicated warehouse. It is useful for exploration, logical serving and querying external files/tables where supported, but repeated high-volume workloads may favor a more persistent serving model.

10.4 Synapse Spark

Synapse Spark pools provide managed Spark. If you already know Databricks, the Spark fundamentals transfer; what changes are platform management, integrations, runtime/features and operating model.

11. Databricks and Snowflake Integration in Azure

11.1 Databricks

A common Azure pattern is ADLS + Databricks + Unity Catalog + Lakeflow + SQL Warehouses, with Entra identities/service principals/managed identities and private networking. ADF or Control-M may remain above platform-local jobs depending on enterprise orchestration.

11.2 Snowflake

Snowflake on Azure can integrate with Azure storage, Entra identity patterns, ADF/Fabric connectors and Power BI. External stages/storage integrations can reach Azure object storage. Keep ownership clear: do not copy data repeatedly between Snowflake and Databricks merely because both exist.

11.3 Cross-platform decision

Architecture decision

Requirement: Domain A transforms in Databricks; Finance consumes in Snowflake.

Recommendation: Publish a governed interface at the narrowest useful boundary and minimize repeated full copies.

Why: Reduces coupling and duplicated compute while respecting platform ownership.

Trade-off: Cross-platform movement adds latency, egress/transfer considerations and reconciliation.

Alternative: Consolidate the workload if platform duplication has no business justification.

12. Power BI and the Serving Layer

Power BI is not just the final chart. Architecture includes semantic model design, measures, relationships, refresh/DirectQuery/Direct Lake-style access where applicable, row-level security, capacity/concurrency, gateway/network access and ownership.

How an architect says it: “I try to push stable business semantics into a governed semantic model rather than letting every report redefine revenue, margin and active customer differently.”

13. Azure Machine Learning - From Notebook to Enterprise Platform

13.1 Why Azure ML exists

Azure ML provides managed capabilities around ML development, compute, environments, data/assets, experiment tracking, pipelines, model registration and deployment. Your Python training code is only one part of that lifecycle.

13.2 Compute Instance vs Compute Cluster

AML compute	Purpose	Think of it as
Compute Instance	personal/interactive development	managed development machine
Compute Cluster	autoscaling job/training/batch compute	ephemeral scalable worker pool
Online Endpoint	low-latency model/API serving	managed real-time inference endpoint
Batch Endpoint	large asynchronous inference	managed batch scoring interface

13.3 Environment

An AML environment defines the software runtime - image/conda/Python dependencies - so jobs can be reproducible across compute. This is similar in spirit to making Databricks job dependencies/runtime explicit, but the AML object model is different.

13.4 Identity and network setup

An AML workspace may need to access storage, registries, Key Vault, databases and model endpoints. The practical troubleshooting sequence remains: compute -> identity -> role/data permission -> network/private DNS -> target. Enterprise workspaces may be private, which is why code can work locally but fail on managed compute or vice versa.

13.5 Pipelines and endpoints

AML pipelines compose repeatable ML steps such as prepare -> train -> evaluate -> register. Online endpoints serve low-latency inference; batch endpoints serve asynchronous bulk inference. Deployment design includes model/environment versioning, traffic rollout, autoscaling, monitoring and rollback.

How an architect says it: "I separate model development from model operations. A successful notebook is not production ML until the environment, data lineage, reproducible pipeline, model version, deployment identity, endpoint scaling and monitoring are defined."

14. Azure OpenAI, AI Search, SharePoint RAG and Agents

14.1 Enterprise RAG project

SharePoint/docs -> Ingest/chunk -> Embeddings -> AI Search -> Azure OpenAI -> App/API

SharePoint is the knowledge source, not the vector database. A production ingestion process extracts authorized documents, normalizes/chunks them, attaches metadata/security information, creates embeddings and indexes content. Retrieval should preserve authorization boundaries so a chatbot does not reveal documents the user cannot access.

14.2 Azure AI Search

Azure AI Search can provide lexical, vector and hybrid retrieval plus filtering/ranking capabilities. Architecture decisions include index schema, chunk size, metadata, vector dimensions/model, hybrid retrieval, semantic/reranking strategy, refresh/deletion and security trimming.

14.3 Azure OpenAI

Azure OpenAI provides managed access to supported OpenAI models within Azure enterprise controls. Production design includes model/deployment choice, token/context limits, quota, latency, content/safety controls, prompt/version management, structured output/tool use, evaluation and cost.

14.4 Agent vs deterministic workflow

Use deterministic code/workflows for known repeatable steps. Use an agent when the task genuinely requires interpretation and dynamic selection among bounded tools. Keep destructive actions behind policy and often human approval.

Architecture decision

Requirement: Data-ops assistant diagnoses failed ADF/Databricks jobs.

Recommendation: Agent reads logs/metadata/runbooks through read-only tools, produces evidence-backed diagnosis; deterministic service performs approved rerun.

Why: Uses AI for ambiguity, keeps production action controlled.

Trade-off: LLM cost/non-determinism and tool security require evaluation/audit.

Alternative: Rules/runbook automation when failure classes are deterministic.

15. Control-M and Cross-Platform Orchestration

Large enterprises often keep Control-M because dependencies span systems: mainframe, file transfer, ADF, Databricks, Snowflake, AML and downstream applications. Platform-native orchestrators remain valuable inside their domain.

Control-M enterprise dependency	- >	ADF ingestion	- >	Databricks/Fabri c transform	- >	Snowflake/BI publish
---------------------------------------	--------	---------------	--------	---------------------------------	--------	-------------------------

How an architect says it: “I use one orchestrator as the owner of each dependency level. Control-M can own the enterprise batch window; ADF owns ingestion steps; Lakeflow owns Databricks-internal tasks. I avoid two schedulers both believing they own the same retry and state.”

15.1 Failure propagation

Define how child failure is returned to Control-M, whether retries happen locally or centrally, how reruns pass business date/publish time, how partial success is recorded, and how downstream publication is prevented until validation passes.

16. Security and Enterprise Guardrails

- **Least privilege:** group/workload identities receive only necessary management/data permissions.
- **Prefer identity over secrets:** Managed Identity/federated credentials where supported; Key Vault for unavoidable secrets/certificates.
- **Private connectivity:** use private endpoints/VNet integration where data classification and enterprise policy require it.
- **Separate environments:** DEV/TEST/PROD identities, resources and deployment approvals.
- **Policy:** Azure Policy can enforce allowed regions, required tags, network/security configurations and other standards.
- **Audit:** centralize activity/resource/service logs with retention appropriate to security and operations.
- **Data protection:** encryption, masking, row/column policies and tokenization where required by the data classification.

17. CI/CD, IaC, Monitoring, Reliability, DR and FinOps

17.1 CI/CD

Application/data code goes through source control, PR checks, automated tests, artifact/package build, environment deployment, integration tests and controlled production promotion. Configuration and secrets are externalized. Production changes are reproducible rather than clicked manually.

17.2 Infrastructure as Code

Bicep/ARM or Terraform can define Azure resources, networking, role assignments and configuration. IaC improves repeatability and reviewability. Not every data object belongs in the same IaC layer; separate platform infrastructure from application/data deployment where lifecycle differs.

17.3 Monitor, Log Analytics and Application Insights

Azure Monitor is the broad monitoring platform. **Log Analytics workspaces** store/query many Azure logs using KQL. **Application Insights** focuses on application performance/telemetry such as requests, dependencies, exceptions and distributed traces.

17.4 Reliability

Use retries with exponential backoff for transient faults, idempotency for replay, dead-letter queues for poison messages, health probes/autoscaling for services, zone/region strategies where required, and runbooks for recovery. Define RPO/RTO per business capability, not one number for the whole platform.

17.5 FinOps

Tag resources by application/domain/environment/owner, attribute capacity/compute/storage/AI costs, detect idle resources, right-size, and measure cost per business workload. Serverless removes server management, not cost responsibility.

18. End-to-End Project Build

18.1 Step 1 - Customer application

Global users enter through Front Door/WAF. API Management authenticates/governs APIs. A customer API runs on App Service or Container Apps. Order transactions use PostgreSQL/Azure SQL. Customer preference/profile documents may use Cosmos DB if the partition/access/global-scale requirements justify it. Redis caches hot non-authoritative data.

18.2 Step 2 - Async integration

Order-created work requiring reliable processing goes to Service Bus. High-volume behavioral events go to Event Hubs. Resource/application notifications can use Event Grid. Functions handle lightweight event actions.

18.3 Step 3 - Enterprise ingestion

On-prem relational sources are reached through self-hosted IR. ADF lands raw extracts into ADLS. API sources use ADF where supported and Functions/containers for custom logic. Event Hubs supplies streaming data. Every source has an owner, contract, replay strategy and audit record.

18.4 Step 4 - Data platform

Databricks processes complex engineering/Delta workloads and publishes curated Customer 360 entities. Fabric can serve integrated Microsoft analytics domains and Power BI-centric use cases. Snowflake domains can consume/publish governed interfaces where organizationally justified. Avoid uncontrolled copies.

18.5 Step 5 - Customer identity resolution

Customer 360 requires matching identities across loyalty ID, account ID, email/phone and household relationships. Exact deterministic keys should be preferred where available; probabilistic/fuzzy matching requires confidence, survivorship rules, manual exception handling and auditability. A golden customer ID is a governed business entity, not just a concatenated key.

18.6 Step 6 - BI and ML

Dimensional/semantic models serve governed BI. Entity-time feature datasets serve churn/next-best-action ML. AML or Databricks ML can train/deploy depending on enterprise platform choice. Do not make the BI star schema the only physical model for ML.

18.7 Step 7 - RAG chatbot

SharePoint policies and FAQs are ingested into AI Search with authorization metadata. Customer-specific facts are retrieved through governed APIs/data tools, not copied into prompts indiscriminately. Azure OpenAI generates responses with evidence. Sensitive actions require deterministic APIs and authorization.

18.8 Step 8 - Operations

Control-M owns cross-enterprise batch dependencies. Platform-local orchestrators own internal DAGs. Azure Monitor/App Insights/Log Analytics plus platform-native telemetry provide end-to-end visibility. CI/CD promotes code/resources. Cost tags/query tags/capacity monitoring support chargeback.

18.9 What fails?

Failure	Signal	Recovery/control
Self-hosted IR unavailable	ADF connectivity/activity failure	HA IR nodes, health alert, retry after recovery

Failure	Signal	Recovery/control
API rate limited	429/retry metrics	backoff, cursor state, controlled concurrency
Event consumer falls behind	Event Hubs lag	scale/optimize consumer, preserve replay window
Cosmos hot partition	RU throttling/skew	revisit key/workload, distribute writes
Private DNS wrong	connection timeout/name resolution	validate DNS zone links/resolution path
Managed Identity lacks data role	403/authorization	least-privilege role correction
RAG retrieves unauthorized doc	security test/audit	security trimming/metadata filter, deny-by-default
Control-M rerun duplicates data	reconciliation anomaly	idempotent business-date/run-key processing

19. Component Selection and Trade-off Playbook

Question	Default reasoning
ADF or Fabric Data Factory?	Existing Azure integration estate vs Fabric-centric analytics strategy; compare connectors, gateway/network, deployment and operations.
ADF or Databricks orchestration?	ADF for heterogeneous connectivity/movement; Lakeflow for Databricks-local data workflows; avoid duplicate ownership.
Function or Container App?	Function for bounded event/serverless function; Container App for longer/custom containerized service/worker.
Container Apps or AKS?	Choose Container Apps unless you need Kubernetes-level platform control/features that justify AKS complexity.
Service Bus or Event Hubs?	Reliable work/message broker vs high-throughput event stream.
Event Grid or Service Bus?	Notification/routing vs durable command/work processing.
Azure SQL or PostgreSQL?	SQL Server/T-SQL ecosystem vs PostgreSQL/open ecosystem; both are relational.
Cosmos or relational DB?	Distributed partitioned low-latency document access vs relational transactions/joins.
ADLS or OneLake?	General Azure lake storage vs Fabric-native tenant storage foundation.
Fabric Lakehouse or Warehouse?	Spark/lakehouse engineering vs SQL-first analytical warehouse.
Synapse Dedicated or Fabric Warehouse?	Existing estate/requirements vs strategic modern Fabric direction; migration is not automatic.
Databricks or Fabric?	Engineering/ML/lakehouse depth vs unified Microsoft SaaS analytics/Power BI integration.
Databricks or Snowflake?	Spark/open lakehouse/ML-heavy vs SQL warehouse/concurrency/operating-model strengths; evaluate actual workload.
Control-M or native scheduler?	Enterprise cross-platform dependency vs platform-local DAG.
Managed Identity or Key Vault secret?	Prefer identity when supported; use Key Vault for secrets that cannot be eliminated.
Front Door or App Gateway?	Global edge HTTP/WAF vs regional VNet HTTP/WAF.
App Gateway or Load Balancer?	Layer-7 HTTP routing vs Layer-4 TCP/UDP.

20. Architect Communication and Interview Answers

20.1 Architecture opening

How an architect says it: “I will start with the business capabilities and non-functional requirements, then work through entry points, application compute, operational stores and integration. After that I will cover the analytical/AI platform. Finally I will explain identity, private networking, deployment, observability, recovery and cost because those cross-cut the entire design.”

20.2 Explain Managed Identity

How an architect says it: “Managed Identity lets an Azure-hosted workload authenticate to another Azure service without storing a client secret in the application. Azure manages the credential lifecycle; I grant that identity only the required RBAC/data permissions. It reduces secret rotation risk, but it does not remove the need for authorization or network connectivity.”

20.3 Explain ADF Integration Runtime

How an architect says it: “Integration Runtime is the execution/connectivity layer ADF uses for data movement and certain activities. Azure IR handles managed cloud connectivity; self-hosted IR is the bridge when ADF must reach private or on-premises systems. For self-hosted IR I design host placement, HA, throughput, patching and network reachability.”

20.4 Explain Customer 360

How an architect says it: “Customer 360 is a governed unified customer capability, not a single database or product. We resolve identities across systems, combine profile, loyalty, transactions and interactions, apply survivorship and quality rules, and expose consumer-specific models for BI, ML and personalization. The key concerns are identity resolution, privacy, freshness, ownership and consistent business meaning.”

20.5 Explain Fabric vs Synapse

How an architect says it: “Synapse remains important because many enterprises have dedicated SQL pools, serverless SQL, Spark and pipelines in production. Fabric is the newer integrated SaaS analytics direction around OneLake and Power BI. I would not migrate simply because Fabric is newer; I would inventory workload behavior, dependencies, performance, governance and cost, then choose a target pattern per workload.”

20.6 Explain Azure ML platform setup

How an architect says it: “I separate the Python model code from the platform it runs on. I identify the AML compute, environment, workload identity, storage/model access, private network path and deployment endpoint. That makes access problems diagnosable: runtime, identity, authorization, DNS/network, then application code.”

20.7 Trade-off answer pattern

Use: Requirement -> constraints -> viable options -> recommendation -> why -> cost/complexity downside -> trigger to revisit.

How an architect says it: “Both options are technically valid. Given the current five-minute freshness SLA and existing ADF estate, I would use incremental ADF ingestion rather than introduce a continuous event architecture. It is simpler to operate and meets the outcome. The trade-off is that it will not support sub-minute reaction; if that becomes a real business requirement, I would revisit Event Hubs/stream processing.”

20.8 Full system-design answer

“For the Customer 360 platform I would separate operational and analytical concerns. Customer-facing traffic enters through Front Door and API Management, then runs on managed application compute. Relational order data stays in

PostgreSQL/Azure SQL; I would only use Cosmos where partitioned low-latency document access justifies it. Service Bus handles reliable asynchronous work and Event Hubs handles high-volume behavioral events. ADF ingests heterogeneous batch/private sources, using self-hosted IR for on-prem connectivity, and lands raw data in ADLS. Databricks or Fabric then builds governed customer, sales and loyalty products; Power BI consumes semantic models and ML consumes entity-time features. For the chatbot, SharePoint content is indexed into AI Search and Azure OpenAI generates grounded answers, with customer-specific actions going through authorized APIs. Across all layers I use Entra identities, Managed Identity where possible, private endpoints/DNS for sensitive services, Key Vault for unavoidable secrets, CI/CD/IaC, centralized monitoring and explicit RPO/RTO. Control-M can remain the enterprise scheduler while each platform owns its internal workflow.”

20.9 Questions an architect should ask before drawing

- What is the business outcome and who consumes it?
- What is the required freshness/latency and what happens if we miss it?
- What are average and peak volume/throughput/concurrency?
- Where does the source live - public cloud, private VNet, on-prem?
- Which identity will the workload use and what exact data-plane permission is required?
- Does the service need private connectivity? Who owns DNS and routing?
- Is the workload request/response, durable messaging, event notification or high-throughput streaming?
- Is the operational data relational or naturally partitioned/document-oriented?
- Which platform owns orchestration and retry state?
- How do we replay/backfill without duplicates?
- How is DEV promoted to PROD?
- What is monitored: technical health, data health, user experience and cost?
- What are RPO/RTO and regional/zone requirements?
- What is the simplest design that meets the requirement?

Final Mental Map



You do not need to memorize every Azure service. You need to recognize the architectural problem category, select a small set of viable services, and explain why one is appropriate under the stated constraints.

Five sentences to remember:

1. My code must run somewhere.
2. That runtime needs an identity and permissions.
3. It also needs a valid network path and DNS.
4. Data movement, processing and serving are different responsibilities.
5. The best architecture is the simplest one that meets the required security, reliability, performance and business outcome.

Currency note: Azure, Fabric and AI service capabilities evolve rapidly. Validate exact current SKUs, feature availability, limits and certification objectives against Microsoft documentation before a production decision or exam.